

Resource Management & Capacity Control

شو يعني؟

يعني انو نحنا بدنا نتحكم بالعمليات التي حيقيم فيها النظام بحيث نلاقي حل وسط يعني ما نخلي استعمال موارد هائل فيينهار النظام ولا نخلي استعمال قليل يبطئ الاستجابة .

كيف بصير هاد الشي؟

كل مرة بصير اتصال بين Laravel و ال db بينفتح خط بين التطبيق و الداتا بيز

يعني مثلا

عنا ١٠٠ مستخدم بنفس اللحظة يعني عنا ١٠٠ اتصال مفتوح سوا فحيكون في :

ضغط كبير عالاداتابيز فاما حيصير بطيء شديد او حينهار الاتصال كلو.

شو الحل؟

بدل ما نفتح اتصال و نسكرو كل مرة : نعيد استخدام نفس الاتصال المفتوح .

و هاد الشي اسمو Connection pool .

مثال مشان تفهمو : تخيلو كل ما بدكم تستخدمو اللابتوب تروحو تشترو لابتوب و تستخدموه و بس تخلصو تبيعوه و اذا بدك متردو تستخدمو اللابتوب تعيدو نفس العملية قديش حيكون فيها عذاب ووقت و كلفة ؟

مثال واقعي :

تخيلو كل مرة واحد بدو يفوت ع مطعم يفتح مطبخ جديد و بس يخلص يسكروه الشي حيسبب كارثة .

وقت نستخدم pool شو بصير ؟

عنا ٤ مطابخ مفتوحة دائما .

كل مطبخ عم يلبي طلب زبون بالتناوب .

هلق حنحتاج بعض القوانين لنحدد كيف بدنا نخصص الموارد

القانون الأول هو لنحسب ال POOL SIZE

```
pool_size = (CPU_cores × 2) + effective_spindle_count
```

CPU_cores : عدد انوية المعالج يعني كم دماغ عندو السيرفر

X 2 : كل نواة تتحمل تقريبا ٢ اتصال شغال

effective_spindle_count: عدد المحركات الفعلية بالقراءة و الكتابة يعني شي بخص التخزين
غالبا بتكون ٠ لو ١

هاد القانون ما حيكون كود حنستخدمو نحنا للحساب فقط

وين طبقناه نحنا فعليا ؟

وقت بدنا نشغل السيرفر بدنا نحدد عدد ال WORKERS اللي بدن يشتغلو

يعني فرضا

PHP_CLI_SERVER_WORKERS = 4 يعني في ٤ WORKERS حيشغلو

يعني في ٤ عمليات حيشغلو بنفس الوقت

Connection Pool (config/database.php)

```
PDO::ATTR_PERSISTENT => true, // إعادة استخدام الاتصالات  
PDO::ATTR_TIMEOUT => 5, // انتظر ٥ ثواني قبل الرفض
```

بدون PERSISTENT، كل طلب بيفتح اتصال جديد بقاعدة البيانات ويبقوله، وهي عملية تستهلك CPU ووقت. مع PERSISTENT بتتعاد الاتصالات و منوفر ال overhead.

```
PDO::MYSQL_ATTR_INIT_COMMAND => 'SET SESSION wait_timeout=600,  
interactive_timeout=600',
```

لو اتصال فتح وصل خامل ١٠ دقائق، MySQL بيقتله تلقائياً. هيك ما بتتراكم اتصالات ميتة تاكل من ال max_connections بدون فايده.

و عالجنا هالشي كمان بقلب سيرفس بحيث اننا استفدنا من REDIS QUEUE

هي مثال من الكود :

```
SendInvoiceJob::dispatch($order->id)->onQueue('invoices');  
SendNotificationJob::dispatch($order->id, $order->user_id)-  
>onQueue('notifications');
```

هاد أقوى شي بالمشروع من ناحية الـ Capacity. بدل ما الـ Worker يضل مشغول ٢-٣ ثواني بإرسال إيميل، بدفع المهمة على Redis Queue و يرجع فوراً. الـ Worker صار جاهز لمستخدم جديد خلال أجزاء من الثانية، والـ Queue Worker منفصل يشتغل بالخلفية.

النتيجة: نفس الـ ٤ Workers يخدموا عشرات الأضعاف من الطلبات